

RFC: A Native qang Module for Qiskit and Cirq

Author : Vicente Humberto Monteverde

Orcid: <https://orcid.org/0000-0001-8884-4811>

Affiliation: UMSA

Country: Argentina

Mail : viny01958@gmail.com/monteverde.5557@umsa.edu

<https://doi.org/10.5281/zenodo.22845682>

19 de sept de 2026 ·

1. Summary and motivation

Qiskit and Cirq both parameterize single-qubit rotations by a raw angle in radians (`RYGate(theta)`, `cirq.ry(theta)`), with no built-in notion of the physically meaningful quantity that angle controls: the Born-rule population bias of the resulting state. The qang unit (qg), published in Monteverde (2026), gives that quantity a name and a closed algebra: $qg_Z = \cos(\theta) = \langle \sigma_z \rangle$ (the "Polar Bias Qang") and $qg_S = H(p_0)$ (the "Entropic Qang", binary Shannon entropy of the outcome distribution), with explicit, tested inversion formulas back to θ .

A reference implementation already exists and runs today: native `RQangGate/FullRQangGate` classes for Qiskit, an equivalent native gate for Cirq, and a `quang.transformations` module (Householder reflections, the Grover diffuser, and Born-rule transition probabilities, all written directly in terms of qg_Z with no runtime trigonometry beyond the algebra's own square root). This has been validated locally against both simulators and, for the Qiskit gate, against IonQ's cloud simulator via `qiskit-ionq`, with results matching theory within ordinary shot noise. This RFC proposes upstreaming a `qang` module into each library: a first-class way to parameterize rotations, transition probabilities, and (per Section 6 below) calibration/estimation by qg value instead of by raw angle, alongside -- not replacing -- the existing angle-based API.

2. Background: the qang (qg) unit

For a real single-qubit state $|\psi(\theta)\rangle = \cos(\theta/2)|0\rangle + \sin(\theta/2)|1\rangle$, θ in $[0, \pi]$, the paper defines two dimensionless units:

Symbol	Definition	Domain Invertible?
--------	------------	--------------------

qg_z (Polar Bias)	$\cos(\text{theta}) = \langle \text{sigma_z} \rangle$	$[-1, 1]$	Yes, everywhere: $\text{theta} = \arccos(\text{qg_z})$
qg_s (Entropic)	$H(p_0)$, binary entropy of p_0	Shannon $= [0, 1]$	Only on a half-domain (theta in $[0, \pi/2]$ or $[\pi/2, \pi]$); 2-to-1 elsewhere

Anchor values from the paper, reproduced by the reference notebook: North Pole $|0\rangle$ has $\text{qg_z} = +1.0$; the equator ($|+\rangle/|-\rangle$) has $\text{qg_z} = 0.0$, $\text{qg_s} = 1.0$; the South Pole $|1\rangle$ has $\text{qg_z} = -1.0$; the canonical T-gate state has $\text{qg_z} = 0.7071$ ($1/\sqrt{2}$).

The paper is explicit (Section 2.3) that qg_z and qg_s are not new physical quantities -- they are named, unit-normalized repackagings of quantities already used throughout the field ($\langle \text{sigma_z} \rangle$, Shannon entropy of measurement outcomes). The proposal here is not to introduce new physics into Qiskit or Cirq, but to give circuit authors a first-class, named, round-trip-safe way to work in this unit instead of writing ad hoc `arccos`/entropy conversions by hand every time -- the same rationale that already justifies `Parameter`, `ParameterExpression`, and unit-carrying types elsewhere in both SDKs.

3. Existing proof of concept

All of the following already runs, in the companion reference notebook and the `quang` GitHub repository (github.com/Viny2030/quang), not just as a design sketch:

- **RQangGate / FullRQangGate (Qiskit).** A `qiskit.circuit.Gate` subclass parameterized directly by qg_z instead of theta , decomposing to a standard `RY` under the hood so it runs on any Qiskit backend, including hardware, once `.decompose()` is called. Verified bit-for-bit against the paper's anchor table on Aer, and cloud-validated against `ionq_simulator` (job 01a0b9af-812b-71c2-a4ea-a90ccb0d09f6, 500 shots/circuit): all four anchor states matched theoretical $P(0)$ within ordinary shot noise ($\sim 2\%$ for $N=500$).
- **A native Cirq gate.** The same construction as a Cirq `Gate`, producing a bit-for-bit identical prepared state to the Qiskit version on the same qg_z input (fidelity 1.000000 in the cross-check).
- **quang.transformations module.** `QangRotation` (the R_y rotation matrix and its induced Born-rule transition-probability matrix, both closed-form functions of qg_z), `householder_qang` (the Householder reflection about a qang-parameterized state), `grover_diffuser_qang` ($= -\text{householder_qang}$, i.e. the Grover diffuser expressed natively in qg-space), and `transition_probability_qang(q1, q2)` (fidelity between two qang states, $[1 + q1*q2 + \sqrt{(1-q1^2)(1-q2^2)}] / 2$). Shipped with 8 passing pytest tests, including a cross-check against literal statevector overlaps for 200 random qg pairs.
- **Shot-noise-aware estimation.** Closed-form delta-method error propagation from measured counts to $\text{qg_z}/\text{theta}$ (Section 6 of the notebook), and, extending it, an exact Bayesian posterior (Section 12; see Section 6 of this RFC) -- both already implemented and validated against Monte Carlo simulation.

What does not yet exist is any of this living *inside* Qiskit or Cirq's own namespaces, discoverable the way `RYGate` or `cirq.ry` are -- today a user must import a third-party `quang` package to get any of it.

4. Proposed Qiskit API surface

A new optional module, `qiskit.circuit.library.qang` (or a `qiskit-qang` extension package if the maintainers prefer to keep the core library's surface area unchanged):

```
from qiskit.circuit.library.qang import RQangGate, Qang

# a rotation gate parameterized by qq_z instead of theta -- decomposes
# to RY
qc.append(RQangGate(qq_z=0.7071), [0])

# a small value type carrying the round-trip conversions, usable wherever
# a plain float/Parameter is accepted today
q = Qang(0.7071, mode="polar")
q.theta      # -> 0.7854 (pi/4)
q.entropic   # -> 0.6009 (qq_S)

# Born-rule transition probability / fidelity between two qang values,
# without constructing circuits at all
from qiskit.circuit.library.qang import transition_probability
transition_probability(1.0, 0.7071)  # -> 0.8536
```

Design constraints this respects: `RQangGate` decomposes to existing basis gates (`RYGate`), so it is transpiler- and hardware-compatible with zero changes elsewhere in Qiskit; `Qang` supports `qiskit.circuit.Parameter/ParameterExpression` binding the same way a raw float does, so it composes with parametrized circuits and QAOA/VQE-style workflows unchanged; and the module adds no new dependency (pure Python + NumPy, matching the existing reference implementation).

Also proposed: `qiskit.quantum_info`-adjacent helpers `householder_qang` and `grover_diffuser_qang`, mirroring the two-line implementations already in `quang.transformations`, since both are one-line derivations once `qq_z` exists as a first-class value and both are already common enough (Grover's algorithm) to justify a named constructor rather than requiring users to hand-derive the matrix.

5. Proposed Cirq API surface

An analogous module, `cirq.qang` (Cirq's convention favors flat top-level namespaces over deep subpackages, so `cirq.qang.RQangGate` or a flattened `cirq.rqang_gate` would both fit existing precedent):

```
import cirq
from cirq.qang import rqang_gate, Qang

# a gate factory parameterized by qq_z -- built from cirq.ry under the
# hood
circuit = cirq.Circuit(rqang_gate(qq_z=0.7071).on(q0))

q = Qang(0.7071, mode="polar")
```

```
q.theta # -> 0.7854
```

The same design constraints apply: the gate decomposes to `cirq.ry` (or is itself defined via `cirq.ry`, matching how the existing reference implementation already builds it), so it is simulator- and hardware-compatible without any change to Cirq's transformer/optimizer passes, and it participates in `cirq.Symbol/sympy`-parameterized circuits the same way a raw angle does today.

Because the reference implementation already demonstrates bit-for-bit identical output between the Qiskit and Cirq versions of this gate (statevector fidelity 1.000000, per Section 3), the two proposed APIs are not just parallel in spirit but numerically interchangeable -- a `qg_z` value means the same physical state regardless of which SDK constructs it, which is itself a small but concrete argument for standardizing the unit across libraries rather than leaving it to each project's own conventions.

6. New theoretical grounding for the API

Four extensions developed in the companion notebook (Sections 8, 9, 11, 12) motivate specific pieces of the proposed API beyond the basic gate:

Quantum Natural Gradient optimizer (Section 8). Raw qg-space gradient descent has a known singularity at the poles (paper Section 4.1: $dE/dqg = -(1/\sin(\theta)) * dE/d\theta$). Preconditioning the step by the inverse Quantum Fisher Information -- $F_Q(qg_Z) = 1/(1-qg_Z^2)$ -- exactly cancels that singularity and converges from a pole at the *same* learning rate as optimizing in θ -space directly, with no clipping or epsilon regularization needed. This motivates shipping a qang-aware natural-gradient step function alongside the gate, for anyone building qg-parameterized variational circuits (VQE/QAOA-style).

Qudit generalization via $\mathfrak{su}(d)$ Cartan generators (Section 9). `qg_z = <sigma_z>` generalizes to the $d-1$ commuting, traceless Hermitian generators of the Cartan subalgebra of $\mathfrak{su}(d)$: `h_k = sqrt(2/(k(k+1))) * diag(1,...,1[k times], -k, 0, ..., 0)`, reducing to exactly `sigma_z` at $d=2$ and to the textbook Gell-Mann pair `(lambda_3, lambda_8)` at $d=3$. Neither Qiskit nor Cirq has native qudit state objects today, so this is proposed as a standalone NumPy reference (density-matrix) module rather than a gate -- future-proofing the qang API for whichever SDK adds qudit support first.

A Haar-motivated default prior (Section 11). For a Haar-random pure state, `qg_z` is exactly `Uniform(-1,1)` (Archimedes' hat-box theorem applied to the Bloch sphere), `theta` has density `sin(theta)/2`, and `qg_s` has a closed-form density diverging at 1. This gives a principled, citable default for any qang API that needs a "no information yet" starting point -- e.g., a default prior for calibration, or a reference distribution for testing gate fidelity across the whole Bloch sphere rather than at a few hand-picked angles.

A Bayesian qang estimator (Section 12). Using the Haar result above as the prior ($p_0 \sim \text{Beta}(1,1)$), the Beta-Binomial posterior for p_0 given k counts out of N shots gives an exact (not linearized) credible interval for θ , via `scipy.stats.beta`. Validated numerically: near a pole (`theta=0.05`, $N=50$, 2000 trials), the standard delta-method

interval's lower bound is pinned at the physical floor $\theta=0$ in 97.4% of trials -- nominally 100% "coverage" that is actually uninformative -- while the Bayesian interval stays genuinely data-driven (e.g. $[0.045, 0.535]$ rad for an all- $|0\rangle$ outcome, vs. the delta-method's $[0.000, 0.277]$). Away from a pole ($\theta=\pi/2$, $N=2000$) the two methods agree to within 0.06% of mean interval width, confirming the Bayesian method is a strict improvement rather than a different answer. Proposed as `qang.Estimator / posterior_theta_credible_interval(k, n_shots, confidence)` in both SDKs, for anyone doing state-tomography-style calibration from shot data.

7. Adoption path

A staged rollout, matching how most gate/utility additions land in both projects:

1. **Standalone package (current state).** `qang` on PyPI, with the Qiskit and Cirq gates as optional integrations (`qang[qiskit]`, `qang[cirq]` extras) so it has zero mandatory dependency on either SDK. This lets the API stabilize and collect real usage before asking either project to take on maintenance burden.
2. **Community/ecosystem listing.** Qiskit maintains an ecosystem page for third-party packages; Cirq has a similar "contrib"-style precedent (e.g. `cirq-google`, `cirq-ionq` as separately-versioned companion packages rather than core). Listing `qang` there is a low-friction next step that requires no code review from either core team.
3. **Upstream RFC / feature request.** Once usage data exists, open a formal proposal against `Qiskit/qiskit` (via a QEP -- Qiskit Enhancement Proposal, their existing RFC mechanism) and `quantumlib/Cirq` (via a GitHub design-doc issue, their existing convention), each scoped to exactly the module described in Sections 4-5 -- deliberately not asking either project to adopt the `qang` terminology project-wide, only to expose it as one more optional, opt-in gate parameterization.
4. **Reference implementation as the PR itself.** Because Sections 3-6 already have working, tested code, the PRs backing each RFC can be close to copy-paste from the existing `qang` module rather than requiring new implementation work -- lowering the review burden asked of either core team.

This document itself is the artifact to attach to steps 3-4: it is written to stand alone (a reviewer needs no access to the private notebook or paper to evaluate the proposal), while the notebook and GitHub repo remain the source of truth for the validated numbers cited throughout.

8. Open questions and risks

- **Naming.** "`qang`"/"`qg`" is a new, non-standard term. Either project's maintainers may reasonably ask why this needs a name at all rather than just exposing `arccos/entropy` helper functions under existing naming (e.g. `RYGate.from_bloch_z_bias`). The RFC should be prepared to make the case on ergonomics (one round-trip-safe value type vs. scattered conversions) rather than on the name itself.
- **Real-hardware validation is still simulator-only.** IonQ validation to date used `ionq_simulator`, not the physical Forte-1 QPU (billed, and not yet run). Section 10 of the notebook is written so the same cell runs unchanged against real

hardware once credit is available; upstream reviewers will likely ask for that hardware run before merging, not just simulator agreement.

- **Qudit support has no home yet.** Section 9's $\text{su}(d)$ generalization has no native SDK gate to attach to in either Qiskit or Cirq today, so it can only ship as a standalone reference module until qudit support exists upstream -- this piece of the proposal may need to wait on a separate, unrelated roadmap item in each project.
- **Duplicate surface area.** Both SDKs already have rich parameter/angle machinery; maintainers may push back on adding a second way to say the same thing. Mitigated by keeping `qang` strictly additive (decomposes to existing gates, never a new basis gate) and clearly scoped as optional.
- **Bayesian estimator scope.** `qang.Estimator` as proposed handles only the single-qubit, single-basis-measurement case validated in Section 12; extending it to multi-qubit or POVM-based calibration (paralleling Section 3-4's mixed-state/POVM work) is unvalidated and out of scope for a first PR.
- **Maintainer bandwidth.** Both projects field many third-party proposals; the staged path in Section 7 is designed to minimize the ask at each step, but there is no guarantee of acceptance at step 3 regardless of code quality -- the standalone `quang` package (step 1) is the fallback that delivers value to users independent of upstream adoption.

9. References

- Monteverde, V. H. "The Qang (qg): A Unified Angular-Probability Unit and Metric for Parametric Quantum Circuit Design." DOI: 10.5281/zenodo.22832150.
- Reference implementation and notebook: github.com/Viny2030/quang.
- Companion notebook, `qang_full_reference_v2.ipynb`: Sections 1-2 (unit definitions, anchor tables), 5 and 7 (existing Qiskit/Cirq gates), 8 (Quantum Natural Gradient), 9 ($\text{su}(d)$ qudit generalization), 10 (IonQ cloud validation, job 01a0b9af-812b-71c2-a4ea-a90ccb0d09f6), 11 (Haar-induced distributions), 12 (Bayesian estimation).